

HONR 2890 Programming Intelligent Robots

Assignment 3: Ball finder

Prof. Christopher Crick

1 Instructions

Write a robot controller that allows a Turtlebot to find a bright yellow ball in its field of vision, and then publishes a message that gives the bearing and distance to the ball.

The first order of business is to detect yellow balls. In your code, the main loop should do the following:

1. Iterate from left to right through the central portion of the image (no need to look at the ceiling), searching for pixel values that fall within the RGB ranges of the yellow ball.
2. Average the x -coordinates (which are the columns, not the rows) of all of the yellow pixels in the image. This is the approximate bearing of the ball.
3. Store the bearing in the BallLocation message. If the ball could not be found in the image, store something that indicates that fact, an invalid number like -1.
4. **Hint:** If looking at every pixel is making your code slow, realize that there is no reason to do so. You can find the center of the yellow object by only looking at every other, or fifth, or tenth pixel. Also, using Numpy operations on the OpenCV matrices is **much** faster than looping, so you can look into that. Numpy is pretty awesome, so I recommend sitting down with a tutorial. It will come in handy not just for this class but forever.

Now that you have the ball location, you can find the distance by subscribing to the `/scan` topic. This will require another handler. The `msg.ranges` field contains one distance measurement (in meters) per $\frac{1}{3}$ degree, in a complete circle around the robot. The camera's field of view occupies 115 of those 1080 measurements, from 220 to 335. *However*, while we count the pixels in an image left-to-right, the measurements in the scan topic go right-to-left (for technical reasons having to do with the orientation of coordinate frame axes). You will have to convert your bearing, which goes from 0 to 250, to an index into the ranges array, which runs from 335 down to 220. **Note:** Some of the range measurements may be invalid. If something is too far away or too close to the camera to be measured (like the support columns on the Turtlebot), it will be marked as not-a-number or as infinity.

Check to make sure the bearing is valid (there exists a ball out there somewhere). If so, store that distance in the BallLocation message.

We publish the BallLocation message and the modified image each time through the main loop, just like your Twist publisher in Assignment 2. You must define the BallLocation message yourself within your package; recall that you did a tutorial on how to do this during the first assignment.

Note: Once you have written your `BallLocation.msg` file, made changes to `package.xml` and `CMakeLists.txt`, and run `colcon build`, you must resource your `~/ros2_ws/install/setup.bash` file so that ROS can find the new message definition.

To determine the appropriate values for yellow ball detection, I recommend outputting an image, with all of the found pixels changed to something obvious, like black. You can then mess with the RGB ranges until you find one that turns parts of the ball black, but does not turn anything else in the lab black. You can also try converting the image to another color space like HSV, which tends to deal with light and shade better than RGB.

2 Sample code

Here is a skeleton for the `ball_detector`. The structure should look familiar.

```
#!/usr/bin/env python3

import rclpy
import cv2
import numpy as np
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data

from cv_bridge import CvBridge, CvBridgeError
from sensor_msgs.msg import Image, LaserScan
from assn3.msg import BallLocation

class Robot(Node):

    def __init__(self):
        super().__init__('ballfinder')
        self.bridge = CvBridge()
        self.image = []
        self.ranges = []
        # You have to create the BallLocation message definition, like you
        # did in the tutorial. It should publish a bearing as an int and
        # a distance (better to call it distance instead of range, because range
        # is a Python reserved word) as a float.
        self.loc_publisher = self.create_publisher(BallLocation, '/ball_location', 10)
        # You should modify the image from the robot's camera so that you can
        # understand what your code is doing. Mask out everything but the
        # yellow pixels. Turn all the yellow pixels chartreuse. Whatever you
        # want. Throw a colored vertical line in that shows what your pixel
        # average is.
        self.im_publisher = self.create_publisher(Image, '/ball_image', 10)
        self.timer = self.create_timer(0.1, self.main_loop)
        self.create_subscription(Image, '/oakd/rgb/preview/image_raw', self.handle_image, qos_...
```

```

        self.create_subscription(LaserScan, '/scan', self.handle_scan, qos_profile_sensor_data)

def handle_image(self, msg):
    try:
        self.raw_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except CvBridgeError:
        print("Unable to convert ROS image to OpenCV format.")

def handle_scan(self, msg):
    self.ranges = msg.ranges

def main_loop(self):

    # If the program is running but we haven't started receiving messages
    # on the ROS topics yet
    if len(self.raw_image) == 0 or len(self.ranges) == 0:
        return
    # Manipulate this one; leave self.raw_image alone. self.raw_image
    # might get overwritten by a new message coming in while we're
    # in the middle of this main_loop.
    image = self.raw_image

    # Locate the bright yellow ball pixels in the image. You don't need
    # to find them all, but you want to make sure you're not finding any
    # non-ball pixels. Find the average column value for the yellow
    # pixels; that will be your bearing to the ball. Manipulate the
    # image however you like to keep track of what you're doing.

    # The laser scanner ranges array contains 1080 values corresponding
    # to a complete 360-degree circle. The camera's field of view is
    # only about 38 degrees, so it corresponds to self.ranges[220:335].
    # There are 250 pixels and only 115 range values, so you'll have to
    # convert. Also, the scan reports measurements from right to left,
    # and your pixel values count from left to right, so it's probably
    # easiest to subtract your converted x average from 335 to get the
    # appropriate index. This will be the distance to the ball.

    try:
        self.im_publisher.publish(self.bridge.cv2_to_imgmsg(image, 'bgr8'))
    except CvBridgeError:
        print("Unable to convert mask to ROS image")

    self.loc_publisher.publish(ball_location)

rcipy.init()

```

```
robot = Robot()
rclpy.spin(robot)
robot.destroy_node()
rclpy.shutdown()
```